Universitas
Esa Unggul
Smart, Creative, and Entrepreneurial

TAHUN
2026

MODUL PRAKTIKUM

SISTEM BASIS DATA TERDISTRIBUSI



Penulis :

Ranny Meilisa, S.Kom., M.Pd.T.

BAHAN AJAR PRAKTIKUM

REPLIKASI MULTI - MASTER (TWO-WAY)

Oleh: Ranny Meilisa, S.Kom., M.Pd.T

Mata Kuliah: Sistem Basis Data Terdistribusi

Jenis Kegiatan: Praktikum berbasis simulasi dan analisis

Perangkat: PostgreSQL dan/atau DBMS lokal, dua skema basis data simulasi

UNIVERSITAS ESA UNGGUL
FAKULTAS ILMU KOMPUTER
PROGRAM STUDI TEKNIK INFORMATIKA

IDENTITAS DAN DESKRIPSI PRAKTIKUM

Mata kuliah	Sistem Basis Data Terdistribusi
Topik	Replikasi Multi-Master (Two-Way)
Sasaran	Mahasiswa S1 Teknik Informatika
Durasi	Disesuaikan dengan kalender akademik
Metode	Demonstrasi, diskusi kasus, praktik SQL, dan analisis hasil
Prasyarat	Mengonfigurasi, menguji, dan mengevaluasi replikasi dua arah antardua instance PostgreSQL

A. Tujuan Praktikum

1. Menjelaskan perbedaan replikasi satu arah dan dua arah
2. Menyiapkan dua instance PostreSQL dengan port berbeda
3. Membagun publication dan subscription di kedua arah
4. Membuktikan perubahan data pada Node A dan Node B tersinkronisasi
5. Mengidentifikasi risiko konflik, replikasi DDL, dan benturan primary key.

B. Dasar Teori

Replikasi multi-master (two-way) memungkinkan kedua node menerima perubahan lokal dan mengirimkan perubahan ke node lain. PostgreSQL tidak menyediakan pengelolaan konflik multi-master secara otomatis melalui logical replication bawaan. Praktikum ini membangun replikasi dua arah untuk lingkungan pembelajaran, bukan sistem multi-master siap produksi.

Pada PostgreSQL 16+, opsi subscription origin = none membantu mencegah perubahan yang berasal dari replikasi diteruskan kembali sehingga tidak terjadi loop. Praktikum menggunakan ID ganjil pada Node A dan ID genap pada Node B untuk mengurangi tabrakan primary key. Pengubahan baris yang sama secara bersamaan tetap berisiko konflik.

Arsitektur: Node A (localhost:5432) ⇄ Node B (localhost:5433). Masing-masing node memiliki publication untuk mengirim perubahan dan subscription untuk menerima perubahan dari node lain.

C. Perangkat dan Prasyarat

1. PostgreSQL versi 16 atau lebih baru; dua instance/cluster aktif pada port 5432 dan 5433 (bisa dua VM atau satu komputer)
2. pgAdmin 4 atau psql; akses superuser untuk konfigurasi laboratorium; akses mengedit postgresql.conf dan pg_hba.conf

3. Backup konfigurasi sebelum mengubah; gunakan jaringan lab terisolasi. Contoh kata sandi wajib diganti dan tidak digunakan di produksi
4. Pastikan kedua database awalnya kosong agar copy_data = false aman digunakan

D. Langkah Kerja Praktikum

1. Menyiapkan dua instance postgreSQL

Komponen	Node A	Node B
Database Server	PostgreSQL	PostgreSQL
Host	localhost	localhost
Port	5432	5433
Username	postgres	postgres
Database	db_node_a	db_node_b
Status	Active	Active

Jalankan dua cluster PostgreSQL terpisah; Node A menggunakan port 5432 dan Node B port 5433. Pada masing-masing postgresql.conf, set nilai berikut lalu restart instance:

```
wal_level = logical
max_replication_slots = 10
max_wal_senders = 10
max_logical_replication_workers = 8
max_worker_processes = 16
```

Pastikan listen_addresses sesuai jaringan lab. Pada pg_hba.conf masing-masing node, izinkan koneksi dari IP node pasangan ke database praktikum memakai scram-sha-256. Contoh khusus lab satu komputer:

```
host    db_multimaster    repl_user    127.0.0.1/32    scram-sha-256
```

Jika menggunakan dua komputer, ganti 127.0.0.1/32 dengan alamat IP node pasangan yang spesifik. Reload konfigurasi pg_hba.conf. Jangan membuka akses ke seluruh internet.

Penjelasan:

1. Node A merupakan instance PostgreSQL pertama
2. Node B merupakan instance PostgreSQL kedua
3. Keduanya menggunakan port berbeda agar tidak terjadi konflik
4. Kedua instance akan dipersiapkan untuk melakukan logical replication

Catatan: PostgreSQL bawaan mendukung logical replication berbasis publication dan subscription. Replikasi dua arah memerlukan konfigurasi tambahan serta pengelolaan konflik data, sehingga belum otomatis aktif hanya dengan menjalankan langkah ini.

2. Membuat Database dan Role pada kedua Node
   Jalankan perintah berikut secara terpisah di koneksi administratif setiap instance. CREATE DATABASE dijalankan di database postgres (bukan di dalam transaction block).

```
CREATE ROLE repl_user WITH LOGIN REPLICATION PASSWORD 'GantiPasswordLab!';
CREATE DATABASE db_multimaster;
```

   Hubungkan ke db_multimaster pada setiap node, lalu berikan akses baca tabel untuk proses initial synchronization/replication:

```
GRANT CONNECT ON DATABASE db_multimaster TO repl_user;
GRANT USAGE ON SCHEMA public TO repl_user;
```

3. Membuat Skema Identik pada Kedua Node
   Jalankan SQL yang sama pada Node A dan Node B:

```
CREATE TABLE produk (
   id BIGINT PRIMARY KEY,
   nama_produk VARCHAR(100) NOT NULL,
   stok INTEGER NOT NULL CHECK (stok >= 0),
   asal_node VARCHAR(10) NOT NULL
);
GRANT SELECT ON TABLE produk TO repl_user;
```

Catatan: primary key wajib untuk contoh UPDATE/DELETE. Skema/DDL tidak otomatis direplikasi; semua perubahan struktur harus diterapkan terkoordinasi di kedua node.

4. Membut Publication pada Masing-masing Node
   Node A, pada db_multimaster:

```
CREATE PUBLICATION pub_a FOR TABLE produk
WITH (publish = 'insert, update, delete');
```

   Node B, pada db_multimaster:

```
CREATE PUBLICATION pub_b FOR TABLE produk
WITH (publish = 'insert, update, delete');
```

5. Membuat Subscription Silang
   Jalankan pada Node A (mengambil perubahan dari Node B):

```
CREATE SUBSCRIPTION sub_from_b
CONNECTION 'host=127.0.0.1 port=5433 dbname=db_multimaster user=repl_user
password=GantiPasswordLab!'
PUBLICATION pub_b
WITH (copy_data = false, origin = none);
```

Jalankan pada Node B (mengambil perubahan dari Node A):

```
CREATE SUBSCRIPTION sub_from_a
CONNECTION 'host=127.0.0.1 port=5432 dbname=db_multimaster user=repl_user
password=GantiPasswordLab!'
PUBLICATION pub_a
WITH (copy_data = false, origin = none);
```

Penting: buat subscription sebagai superuser di lingkungan lab. CREATE SUBSCRIPTION tidak dijalankan dalam transaction block bila membuat slot secara otomatis. Penggunaan password dalam conninfo hanya untuk demonstrasi; gunakan pengelolaan kredensial yang lebih aman di lingkungan nyata.

6. Menguji INSERT Dua Arah
   Node A: gunakan ID ganjil. Node B: gunakan ID genap. Hindari menulis ID sama pada kedua node.

```
-- Node A
INSERT INTO produk VALUES (101, 'Keyboard', 20, 'A');
INSERT INTO produk VALUES (103, 'Mouse', 35, 'A');

-- Node B
INSERT INTO produk VALUES (102, 'Monitor', 10, 'B');
INSERT INTO produk VALUES (104, 'Headset', 15, 'B');
```

Tunggu beberapa detik, lalu pada kedua node jalankan:

```
SELECT id, nama_produk, stok, asal_node FROM produk ORDER BY id;
```

Hasil yang diharapkan pada KEDUA node: ID 101, 102, 103, 104 muncul dengan isi yang sama. Replikasi bersifat asinkron; hasil tidak harus muncul seketika.

7. Menguji UPDATE dan DELETE Dua Arah

```
-- Node A: ubah data yang berasal dari A
UPDATE produk SET stok = 25 WHERE id = 101;

-- Node B: ubah data yang berasal dari B
UPDATE produk SET stok = 12 WHERE id = 102;
DELETE FROM produk WHERE id = 104;

-- Jalankan pada kedua node setelah sinkron
SELECT * FROM produk ORDER BY id;
```

Hasil yang diharapkan: stok ID 101 = 25; stok ID 102 = 12; ID 104 hilang pada kedua node. Jangan mengubah baris yang sama secara simultan selama pengujian dasar.

8. Memantau Status Replikasi

```
-- Pada masing-masing node: status subscription lokal
SELECT subname, pid, received_lsn, latest_end_lsn, latest_end_time
FROM pg_stat_subscription;

-- Pada masing-masing node: aktivitas pengirim (jika aktif)
SELECT application_name, state, sent_lsn, replay_lsn
FROM pg_stat_replication;
```

Interpretasi: pid yang terisi pada pg_stat_subscription biasanya menunjukkan worker aktif; periksa log PostgreSQL dan latensi jika data belum tiba. Nama kolom/hasil dapat berbeda menurut versi PostgreSQL.

E. Tabel Pengamatan

Uji	Perubahan	Hasil Node A	Hasil Node B
1	INSERT 101 dari A	.....	.....
2	INSERT 102 dari B	.....	.....
3	UPDATE 101 dari A	.....	.....
4	UPDATE 102 dari B	.....	.....
5	DELETE 104 dari B	.....	.....

F. Troubleshooting dan Batasan

Gejala	Pemeriksaan / solusi
Connection refused	Periksa service, port 5432/5433, listen_addresses dan firewall.
No pg_hba.conf entry / password gagal	Periksa rule pg_hba.conf, IP, role, password dan metode autentikasi.
Logical decoding requires wal_level logical	Set wal_level = logical lalu restart kedua node.
Duplicate key / apply worker berhenti	Pastikan ID unik lintas node; periksa log; selesaikan konflik secara manual sebelum melanjutkan.
Data tidak muncul	Periksa publication, subscription, worker, hak SELECT, dan log. copy_data=false tidak menyalin data lama.
Perubahan struktur tabel tidak ikut	Jalankan DDL kompatibel di kedua node secara terkoordinasi.

Batasan penting: logical replication bawaan bukan solusi multi-master bebas konflik. UPDATE bersamaan pada baris sama, DELETE versus UPDATE, perubahan sequence, dan konflik unik

memerlukan desain penanganan khusus. Praktikum tidak menjamin konsistensi kuat atau ketersediaan tinggi.

G. Latihan Mandiri

1. Tambahkan minimal 3 produk baru dari Node A (ID ganjil) dan 3 produk dari Node B (ID genap). Tampilkan hasil akhir pada kedua node.
2. Lakukan dua UPDATE dari node yang berbeda pada baris yang berbeda. Dokumentasikan waktu propagasi secara sederhana.
3. Simulasikan Node B berhenti sementara, lakukan INSERT pada Node A, hidupkan Node B, lalu amati proses penyusulan data.
4. Jelaskan mengapa copy_data = false dipilih untuk tabel awal kosong dan apa risikonya bila tabel sudah berisi data.
5. Analisis skenario kedua node menulis ID 200 secara bersamaan (cukup simulasi konseptual; jangan merusak konfigurasi utama).

H. Evaluasi

1. Apa perbedaan replikasi primary–replica dengan replikasi two-way?
2. Jelaskan fungsi publication, subscription, dan replication slot !
3. Mengapa opsi origin = none relevan untuk replikasi dua arah pada PostgreSQL 16+?
4. Apa penyebab konflik primary key dan bagaimana strategi pencegahannya?
5. Mengapa logical replication bawaan tidak otomatis menjadi sistem multi-master siap produksi?

I. Rubrik Penilaian

Aspek	Bobot
Konfigurasi dua instance dan keamanan dasar	20%
Publication dan subscription dua arah berfungsi	30%
Pengujian INSERT/UPDATE/DELETE dan bukti	25%
Analisis kendala, konflik, dan batasan	15%
Kerapian laporan dan kesimpulan	10%